

Research Copilot: Academic Paper Assistant

Assignment Instructions - Detailed Guide

Table of Contents

1. [Project Overview](#)
 2. [Learning Objectives](#)
 3. [Project Scope and Paper Selection](#)
 4. [Technical Stack](#)
 5. [System Architecture](#)
 6. [RAG Pipeline Implementation](#)
 7. [Prompt Engineering Strategies](#)
 8. [Website and User Interface](#)
 9. [Repository Structure](#)
 10. [Video Deliverable](#)
 11. [Deliverables Checklist](#)
 12. [Evaluation Rubric](#)
 13. [Appendix: Resources and Examples](#)
-

1. Project Overview

1.1 Description

Research Copilot is a conversational AI assistant that helps researchers, students, and professionals interact with academic literature. You will build a system that can answer questions about a collection of 20 academic papers from your field of study using Retrieval-Augmented Generation (RAG).

The system must be capable of:

- Understanding and extracting information from PDF documents
- Answering complex questions by retrieving relevant passages
- Providing accurate citations in APA format
- Maintaining conversation context across multiple exchanges

- Presenting information through an intuitive, feature-rich web interface

1.2 Why This Project Matters

In your professional career, you will need to:

- Quickly understand large volumes of research literature
- Synthesize information from multiple sources
- Build AI-powered tools that augment human capabilities
- Understand the strengths and limitations of Large Language Models (LLMs)

This project gives you hands-on experience with production-ready AI systems.

2. Learning Objectives

By completing this project, you will:

Objective	Skills Developed
RAG Architecture	Document ingestion, chunking, embeddings, vector databases, retrieval
Prompt Engineering	Clear instructions, structured outputs, few-shot learning, chain-of-thought
API Integration	Working with OpenAI GPT-4 API, managing tokens and costs
Web Development	Building interactive UIs with Streamlit/Gradio, user experience design
Software Engineering	Code organization, documentation, version control, reproducibility
Research Skills	Literature review, academic paper analysis, citation management

3. Project Scope and Paper Selection

3.1 Paper Requirements

Each student must select 20 academic papers related to their field of study or career interests.

Selection Guidelines:

Criterion	Requirement
Quantity	Exactly 20 papers
Format	PDF format only
Language	English (primary), Spanish (acceptable)
Recency	At least 10 papers from the last 5 years (2020-2025)
Diversity	Mix of seminal works and recent research
Quality	Peer-reviewed journals or top conferences preferred
Length	Minimum 5 pages per paper

Field Examples:

Career/Field	Suggested Topics
Computer Science	Machine Learning, NLP, Computer Vision, Cybersecurity
Medicine	Clinical AI, Drug Discovery, Medical Imaging
Business	FinTech, Marketing Analytics, Operations Research

Engineering	Robotics, IoT, Sustainable Energy
Psychology	Cognitive Science, Behavioral AI, Human-Computer Interaction
Law	Legal Tech, AI Regulation, Privacy Laws

3.2 Paper Documentation

Create a file `papers/paper_catalog.json` with metadata for each paper:

```
{
  "papers": [
    {
      "id": "paper_001",
      "title": "Attention Is All You Need",
      "authors": ["Vaswani, A.", "Shazeer, N.", "Parmar, N."],
      "year": 2017,
      "venue": "NeurIPS",
      "doi": "10.48550/arXiv.1706.03762",
      "filename": "vaswani2017attention.pdf",
      "topics": ["transformers", "attention mechanism", "NLP"],
      "abstract": "The dominant sequence transduction models..."
    }
  ]
}
```

4. Technical Stack

4.1 Required Technologies

Category	Technology	Purpose
Language	Python 3.10+	Primary programming language
LLM API	OpenAI GPT-4 API	Chat completions and reasoning
Embeddings	OpenAI text-embedding-3-small	Document and query embeddings

Vector Database	ChromaDB	Storage and retrieval of embeddings
-----------------	----------	-------------------------------------

4.2 Recommended Open Source Packages

PDF Processing

PyMuPDF (fitz) - Fast PDF text extraction
pdfplumber - Table extraction from PDFs
pypdf - PDF manipulation

Text Processing

tiktoken - OpenAI tokenizer (required)
langchain - RAG pipeline orchestration (optional)
llama-index - Alternative RAG framework (optional)

Vector Database Options

chromadb - Recommended (simple, local)
faiss-cpu - Facebook's similarity search
qdrant-client - Alternative vector DB

Web Framework (Choose One)

streamlit - Recommended for rapid development
gradio - Alternative with good ML integration
flask - For custom designs (more work)

Utilities

python-dotenv - Environment variable management
pandas - Data manipulation
tqdm - Progress bars
loguru - Logging
pytest - Testing

4.3 Installation Requirements

Create a `requirements.txt` file:

```
# Core
openai>=1.12.0
tiktoken>=0.5.0
chromadb>=0.4.0

# PDF Processing
PyMuPDF>=1.23.0
pdfplumber>=0.10.0

# Web Interface
streamlit>=1.31.0

# Utilities
python-dotenv>=1.0.0
pandas>=2.0.0
tqdm>=4.66.0
loguru>=0.7.0

# Development
pytest>=7.4.0
black>=24.0.0
```

4.4 API Keys and Environment Setup

Create a `.env` file (DO NOT commit to GitHub):

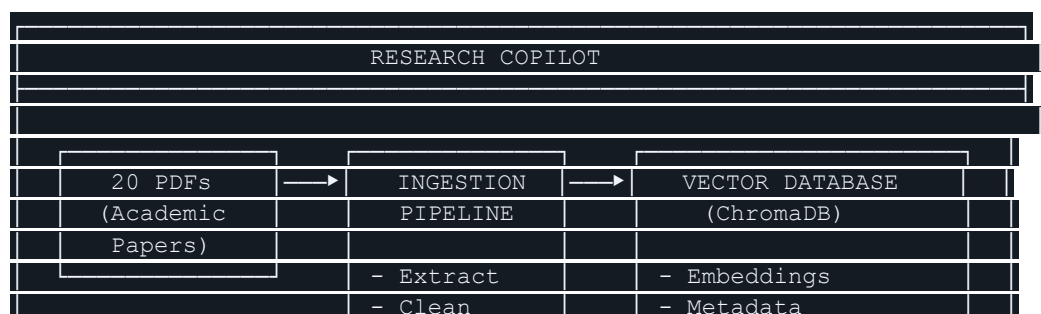
```
OPENAI_API_KEY=sk-your-api-key-here
```

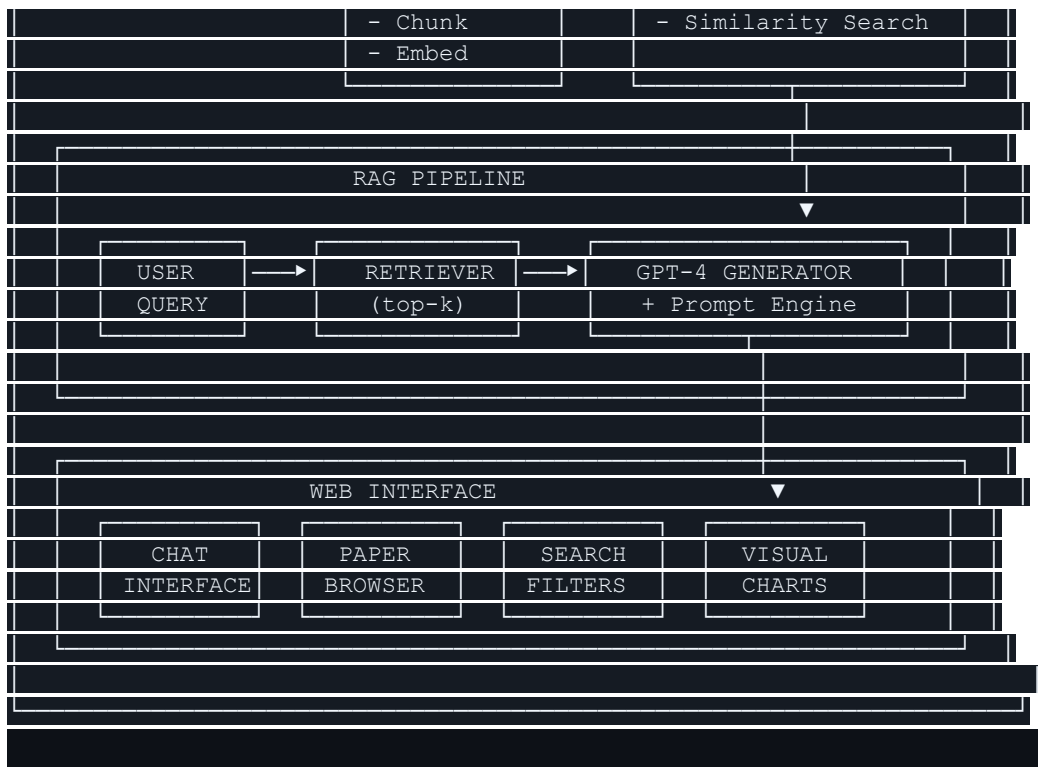
Create a `.env.example` file (commit this):

```
OPENAI_API_KEY=sk-your-api-key-here
```

5. System Architecture

5.1 High-Level Architecture





5.2 Component Breakdown

Component	Responsibility	Key Files
Ingestion	PDF parsing, text cleaning, metadata extraction	<code>src/ingestion/</code>
Chunking	Split documents into retrievable units	<code>src/chunking/</code>
Embedding	Generate vector representations	<code>src/embedding/</code>
Vector Store	Store and retrieve similar chunks	<code>src/vectorstore/</code>
Retrieval	Find relevant chunks for queries	<code>src/retrieval/</code>
Generation	Produce answers using GPT-4	<code>src/generation/</code>
Prompts	Prompt templates and strategies	<code>prompts/</code>
Web UI	User interface and visualization	<code>app/</code>

6. RAG Pipeline Implementation

6.1 Document Ingestion

Step 1: PDF Text Extraction

```
# src/ingestion/pdf_extractor.py
import fitz  # PyMuPDF

def extract_text_from_pdf(pdf_path: str) -> dict:
    """
    Extract text and metadata from a PDF file.

    Returns:
        dict with keys: text, metadata, pages, extraction_warnings
    """
    doc = fitz.open(pdf_path)

    full_text = ""
    pages = []
    warnings = []

    for page_num, page in enumerate(doc):
        text = page.get_text()
        pages.append({
            "page_number": page_num + 1,
            "text": text,
            "char_count": len(text)
        })
        full_text += f"\n[PAGE {page_num + 1}]\n{text}"

    # Document metadata
    metadata = doc.metadata

    return {
        "text": full_text,
        "metadata": metadata,
        "pages": pages,
        "total_pages": len(doc),
        "extraction_warnings": warnings
    }
```

Step 2: Text Cleaning

Handle common PDF extraction issues:

```
# src/ingestion/text_cleaner.py
import re
```



```
def clean_extracted_text(text: str) -> str:
    """Clean and normalize extracted PDF text."""

    # Remove excessive whitespace
    text = re.sub(r'\s+', ' ', text)

    # Fix hyphenated words at line breaks
    text = re.sub(r'(\w+)-\s+(\w+)', r'\1\2', text)

    # Remove page numbers and headers (customize per document)
    text = re.sub(r'\n\d+\n', '\n', text)

    # Normalize quotes
    text = text.replace('"', "'").replace("'", '"')

    return text.strip()
```

Extraction Limitations to Document

You MUST document the following limitations in your README:

Limitation	Impact	Mitigation
Tables	Often extracted as unstructured text	Use <code>pdfplumber</code> for table-heavy documents
Formulas	Mathematical notation may be garbled	Note in metadata, consider OCR alternatives
Images	Image content not extracted	Document which figures are missing
Multi-column	Text order may be incorrect	Manual review of key sections
Scanned PDFs	No text layer	Use OCR (Tesseract) if needed

6.2 Chunking Strategies

REQUIRED: Implement at least 2 chunking configurations

Configuration 1: Small Chunks (256 tokens)

- Better for precise, factual questions
- Higher retrieval accuracy for specific details
- More chunks = more API calls for context

Configuration 2: Large Chunks (1024 tokens)

- Better for complex, multi-part questions
- More context per chunk
- Fewer chunks = lower cost

Implementation:

```
# src/chunking/chunker.py
import tiktoken

class TokenChunker:
    def __init__(
        self,
        chunk_size: int = 512,
        chunk_overlap: int = 50,
        model: str = "gpt-4"
    ):
        self.chunk_size = chunk_size
        self.chunk_overlap = chunk_overlap
        self.encoder = tiktoken.encoding_for_model(model)

    def count_tokens(self, text: str) -> int:
        """Count tokens in text."""
        return len(self.encoder.encode(text))

    def chunk_text(self, text: str, metadata: dict = None) -> list[dict]:
        """
        Split text into overlapping chunks.

        Returns:
            List of chunk dictionaries with text and metadata
        """
        tokens = self.encoder.encode(text)
        chunks = []

        start = 0
        chunk_id = 0

        while start < len(tokens):
            end = start + self.chunk_size
            chunk_tokens = tokens[start:end]
            chunk_text = self.encoder.decode(chunk_tokens)

            chunks.append({
                "chunk_id": chunk_id,
                "text": chunk_text,
                "token count": len(chunk_tokens),
                "start_token": start,
```

```
        "end_token": end,
        "metadata": metadata or {}
    })

    start += self.chunk_size - self.chunk_overlap
    chunk_id += 1

    return chunks
```

Chunking Comparison Table (Include in README)

Configuration	Chunk Size	Overlap	Total Chunks	Use Case
Small	256 tokens	25 tokens	~2000	Factual Q&A
Medium	512 tokens	50 tokens	~1000	Balanced
Large	1024 tokens	100 tokens	~500	Complex reasoning

6.3 Embedding Generation

Use OpenAI's embedding model:

```
# src/embedding/embedder.py
from openai import OpenAI
from typing import List

class OpenAIEmbedder:
    def __init__(self, model: str = "text-embedding-3-small"):
        self.client = OpenAI()
        self.model = model

    def embed_texts(self, texts: List[str]) -> List[List[float]]:
        """Generate embeddings for a list of texts."""
        response = self.client.embeddings.create(
            model=self.model,
            input=texts
        )
        return [item.embedding for item in response.data]

    def embed_query(self, query: str) -> List[float]:
        """Generate embedding for a single query."""
        return self.embed_texts([query])[0]
```

Embedding Model Options

Model	Dimensions	Cost	Quality
<code>text-embedding-3-small</code>	1536	\$0.02/1M tokens	Good
<code>text-embedding-3-large</code>	3072	\$0.13/1M tokens	Best
<code>text-embedding-ada-002</code>	1536	\$0.10/1M tokens	Legacy

Recommended: `text-embedding-3-small` for best cost-quality balance.

6.4 Vector Storage with ChromaDB

```
# src/vectorstore/chroma_store.py
import chromadb
from chromadb.config import Settings

class ChromaVectorStore:
    def __init__(self, persist_directory: str = "./chroma_db"):
        self.client = chromadb.PersistentClient(
            path=persist_directory,
            settings=Settings(anonymized_telemetry=False)
        )
        self.collection = None

    def create_collection(self, name: str):
        """Create or get a collection."""
        self.collection = self.client.get_or_create_collection(
            name=name,
            metadata={"hnsw:space": "cosine"}
        )

    def add_documents(
        self,
        ids: list[str],
        documents: list[str],
        embeddings: list[list[float]],
        metadatas: list[dict]
    ):
        """Add documents with embeddings and metadata."""
        self.collection.add(
            ids=ids,
            documents=documents,
            embeddings=embeddings,
            metadatas=metadatas
        )

    def query(
        self,
        query_embedding: list[float],
        n_results: int = 5,
```

```

        where: dict = None
    ) -> dict:
        """Query similar documents."""
        return self.collection.query(
            query_embeddings=[query_embedding],
            n_results=n_results,
            where=where,
            include=["documents", "metadatas", "distances"]
        )

```

6.5 Retrieval with Metadata

Each retrieved chunk must include:

```

# Example retrieved chunk structure
{
    "chunk_id": "paper_001_chunk_015",
    "text": "The transformer architecture relies on...",
    "metadata": {
        "paper_id": "paper_001",
        "paper_title": "Attention Is All You Need",
        "authors": "Vaswani et al.",
        "year": 2017,
        "page_number": 3,
        "section": "Architecture"
    },
    "similarity_score": 0.89
}

```

7. Prompt Engineering Strategies

REQUIRED: Implement and Test 4 Strategies

Strategy 1: Clear Instructions with Delimiters

```
SYSTEM_PROMPT_V1 = """
```

```
You are Research Copilot, an expert academic assistant.
```

```
YOUR TASK:
```

```
Answer questions about academic papers using ONLY the provided context.
```

```
RULES:
```

1. Base your answer ONLY on the provided context
2. If the context doesn't contain enough information, say "I cannot find this information in the provided papers"
3. Always cite your sources using APA format
4. Be precise and academic in your tone

```
CONTEXT:
###
{context}
###
```

```
USER QUESTION: {question}
```

```
YOUR ANSWER:
"""
```

Strategy 2: Structured JSON Output

```
SYSTEM_PROMPT_V2 = """
```

```
You are Research Copilot. Answer questions about academic papers.
```

```
You must respond in the following JSON format:
```

```
{
  "answer": "Your detailed answer here",
  "confidence": "high|medium|low",
  "citations": [
    {
      "paper": "Paper title",
      "authors": "Author names",
      "year": 2023,
      "quote": "Relevant quote from paper"
    }
  ],
  "related_topics": ["topic1", "topic2"]
}
```

```
CONTEXT:
{context}
```

```
QUESTION: {question}
```

```
"""
```

Strategy 3: Few-Shot Learning

```
SYSTEM_PROMPT_V3 = """
```

```
You are Research Copilot. Here are examples of how to answer questions:
```

```
EXAMPLE 1:
```

```
Question: What is the main contribution of the transformer paper?
```

```
Context: "We propose a new simple network architecture, the Transformer, based solely on attention mechanisms..." (Vaswani et al., 2017, p. 1)
```

```
Answer: The main contribution of the transformer paper is proposing a novel neural network architecture that relies entirely on attention mechanisms, eliminating the need for recurrence and convolutions. According to Vaswani et al. (2017), "We propose a new simple network architecture, the Transformer, based solely on attention mechanisms" (p. 1).
```

```
EXAMPLE 2:
```

```
Question: How does BERT handle bidirectional context?
```

```
Context: "BERT is designed to pre-train deep bidirectional representations by jointly conditioning on both left and right context in all layers." (Devlin et al., 2019, p. 2)
Answer: BERT handles bidirectional context through its pre-training strategy. As Devlin et al. (2019) explain, the model "jointly condition[s] on both left and right context in all layers" (p. 2), allowing it to build deep bidirectional representations.
```

```
---
Now answer the following:
```

```
CONTEXT:
{context}
```

```
QUESTION: {question}
```

Strategy 4: Chain-of-Thought Reasoning

```
SYSTEM PROMPT V4 = ""
You are Research Copilot. For complex questions, think step by step.
```

```
CONTEXT:
{context}
```

```
QUESTION: {question}
```

- Think through this step-by-step:
1. First, identify what the question is asking
 2. Find relevant information in the context
 3. Connect the pieces of information
 4. Formulate a comprehensive answer with citations

```
STEP-BY-STEP REASONING:
[Your reasoning here - this will not be shown to user]
```

```
FINAL ANSWER:
[Your answer here with APA citations]
""
```

```
# Note: In implementation, you may want to hide the reasoning
# and only show the final answer to users
```

Prompt Comparison Requirements

You must create a comparison table in your README:

Strategy	Best For	Latency	Token Usage	Citation Quality

V1: Delimiters	Simple factual Q&A	Low	Low	Medium
V2: JSON	API integration	Medium	Medium	High
V3: Few-shot	Consistent formatting	Medium	High	High
V4: Chain-of-thought	Complex reasoning	High	High	High

8. Website and User Interface

8.1 Required Features

Your web application must include ALL of the following:

Feature 1: Chat Interface

- Real-time conversation with the assistant
- Message history with user/assistant differentiation
- Typing indicators
- Clear conversation button

Feature 2: Paper Browser

- List all 20 papers with metadata
- Search papers by title, author, year
- View paper details and abstracts
- Filter by topic/category

Feature 3: Citation Display

- Show sources for each answer
- APA format citations
- Link to specific paper and page
- Highlight relevant quotes

Feature 4: Visualization Dashboard

- Number of papers by year
- Topic distribution chart
- Query history statistics
- Token usage tracking

Feature 5: Search Filters

- Filter by paper
- Filter by date range
- Filter by topic
- Filter by author

8.2 Design Requirements

Visual Design

- Professional, academic aesthetic
- Field-specific color scheme or theme
- Responsive layout
- Dark/light mode (optional but recommended)

User Experience

- Clear navigation
- Loading states for all async operations
- Error messages that help users
- Keyboard shortcuts (optional)

8.3 Streamlit Implementation Structure

```
app/
├── main.py          # Main Streamlit app
├── pages/
│   ├── 1_Chat.py    # Chat interface
│   ├── 2_Papers.py  # Paper browser
│   ├── 3_Analytics.py # Visualizations
│   └── 4_Settings.py # Configuration
├── components/
│   ├── chat_message.py # Chat message component
│   ├── paper_card.py   # Paper display card
│   └── citation.py     # Citation formatter
└── utils/
    ├── session.py     # Session state management
    └── styling.py     # Custom CSS
```

Example Main App Structure

```
# app/main.py
import streamlit as st
from src.rag_pipeline import RAGPipeline
from src.vectorstore import ChromaVectorStore

st.set_page_config(
    page_title="Research Copilot",
    page_icon="📖",
    layout="wide",
    initial_sidebar_state="expanded"
)

# Custom CSS for academic theme
st.markdown("""
<style>
    .main-header {
        font-family: 'Georgia', serif;
        color: #1a1a2e;
    }
    .citation-box {
        background-color: #f0f0f5;
        border-left: 4px solid #4a4a8a;
        padding: 10px;
        margin: 10px 0;
    }
</style>
""", unsafe_allow_html=True)

# Initialize session state
if "messages" not in st.session_state:
    st.session_state.messages = []

if "rag_pipeline" not in st.session_state:
    st.session_state.rag_pipeline = RAGPipeline()

# Sidebar
with st.sidebar:
    st.image("assets/logo.png", width=150)
    st.title("Research Copilot")
    st.markdown("Your AI research assistant")

    # Paper filter
    selected_papers = st.multiselect(
        "Filter by papers:",
        options=get_all_paper_titles()
    )

    # Prompt strategy selector
    strategy = st.selectbox(
        "Prompt Strategy:",
        ["Standard", "JSON Output", "Few-Shot", "Chain-of-Thought"]
    )
```

```

# Main chat area
st.header("💬 Chat with your papers")

# Display chat history
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])
        if "citations" in message:
            display_citations(message["citations"])

# User input
if prompt := st.chat_input("Ask a question about your papers..."):
    # Add user message
    st.session_state.messages.append({"role": "user", "content": prompt})

    # Get response
    with st.spinner("Searching papers..."):
        response, citations = st.session_state.rag_pipeline.query(
            prompt,
            strategy=strategy,
            filter_papers=selected_papers
        )

    # Add assistant message
    st.session_state.messages.append({
        "role": "assistant",
        "content": response,
        "citations": citations
    })

st.rerun()

```

8.4 Running Locally

Setup Instructions

```

# Clone the repository
git clone https://github.com/your-username/research-copilot.git
cd research-copilot

# Create virtual environment
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Set up environment variables
cp .env.example .env
# Edit .env and add your OpenAI API key

# Run ingestion (first time only)
python src/ingest.py

# Start the application

```

```
streamlit run app/main.py
```

Expected Output

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8501>

Network URL: <http://192.168.1.100:8501>

9. Repository Structure

9.1 Required Structure

```
research-copilot/
|
|-- README.md                # Main documentation (REQUIRED)
|-- requirements.txt         # Python dependencies (REQUIRED)
|-- .env.example             # Environment template (REQUIRED)
|-- .gitignore               # Git ignore file (REQUIRED)
|
|-- papers/                  # Academic papers
|   |-- paper_catalog.json   # Paper metadata
|   |-- paper_001.pdf
|   |-- paper_002.pdf
|   |-- ... (20 papers)
|
|-- src/                     # Source code
|   |-- __init__.py
|   |-- ingestion/
|   |   |-- __init__.py
|   |   |-- pdf_extractor.py
|   |   |-- text_cleaner.py
|   |-- chunking/
|   |   |-- __init__.py
|   |   |-- chunker.py
|   |-- embedding/
|   |   |-- __init__.py
|   |   |-- embedder.py
|   |-- vectorstore/
|   |   |-- __init__.py
|   |   |-- chroma_store.py
|   |-- retrieval/
|   |   |-- __init__.py
|   |   |-- retriever.py
|   |-- generation/
|   |   |-- __init__.py
|   |   |-- generator.py
|   |-- rag_pipeline.py      # Main pipeline orchestration
|
```

```

├── prompts/                                # Prompt templates
│   ├── v1_delimiters.txt
│   ├── v2_json_output.txt
│   ├── v3_few_shot.txt
│   └── v4_chain_of_thought.txt
├──
├── app/                                    # Web application
│   ├── main.py
│   ├── pages/
│   └── components/
├──
├── eval/                                  # Evaluation
│   ├── questions.json                    # Test questions
│   ├── evaluate.py                       # Evaluation script
│   └── results/                          # Evaluation results
├──
├── demo/                                  # Demo materials
│   ├── screenshots/
│   └── demo_video.md                     # Link to video
├──
└── tests/                                # Unit tests
    ├── test_ingestion.py
    ├── test_chunking.py
    └── test_retrieval.py

```

9.2 README.md Requirements

Your README must include:

Section 1: Project Title and Description

- Clear project name
- One-paragraph description
- Your field/topic of papers

Section 2: Features

- List of all features
- Screenshots

Section 3: Architecture

- System diagram
- Component explanation

Section 4: Installation

- Step-by-step setup instructions

- Must work with single command

Section 5: Usage

- How to run the application
- Example queries

Section 6: Technical Details

- Chunking configurations comparison
- Prompt strategies comparison
- Embedding model used
- Token usage estimates

Section 7: Evaluation Results

- Metrics and performance
- Comparison table

Section 8: Limitations

- At least 3 documented limitations
- Suggestions for future improvement

Section 9: Author Information

- Your name
- Course information
- Date

10. Video Deliverable

10.1 Video Requirements

Aspect	Requirement
Duration	3-5 minutes maximum

Format	MP4 or upload to YouTube (unlisted)
Resolution	Minimum 720p
Audio	Clear narration required

10.2 Video Structure

Part 1: Introduction (0:00 - 0:30)

- Your name and field of study
- Brief description of your paper collection
- What problem Research Copilot solves

Part 2: Architecture Overview (0:30 - 1:00)

- Show your system architecture diagram
- Briefly explain each component
- Mention key technologies used

Part 3: Live Demo (1:00 - 3:00)

- Show the web interface
- Ask at least 3 different types of questions:
 - Simple factual question
 - Complex question requiring multiple sources
 - Question that requires the system to say "I don't know"
- Show the citation display
- Demonstrate paper browser and filters
- Show any visualizations

Part 4: Technical Discussion (3:00 - 4:00)

- Compare prompt strategies (show results)
- Discuss chunking configuration impact
- Show any metrics or evaluation results

Part 5: Conclusions (4:00 - 5:00)

- What you learned
- 3 limitations of your system

- 3 potential improvements
- Final thoughts

10.3 Recording Tips

- Use screen recording software (OBS, Loom, or built-in OS tools)
- Prepare your demo in advance - know what questions to ask
- Speak clearly and at moderate pace
- Show your face briefly at intro/conclusion (optional but recommended)
- Edit out any long loading times

10.4 Submission

Include in your repository:

```
demo/  
├── video_link.md    # YouTube/Google Drive link  
├── screenshots/  
│   ├── chat_interface.png  
│   ├── paper_browser.png  
│   └── analytics.png
```

11. Deliverables Checklist

Use this checklist before submission:

Repository

README.md with all required sections
requirements.txt with all dependencies
.env.example file
.gitignore excluding .env, pycache, etc.
All source code in src/
All 20 papers in papers/
Paper catalog JSON file
All 4 prompt versions in prompts/
Web application in app/
Evaluation questions and script

Code Quality

- Code runs with single command as documented
- No hardcoded API keys
- Functions have docstrings
- Consistent code style
- Error handling implemented

Functionality

- All 20 papers are indexed
- Chat interface works
- Citations are displayed in APA format
- Paper browser shows all papers
- Filters work correctly
- At least one visualization
- Conversation memory works

Documentation

- Chunking configurations documented
- Prompt strategies compared
- Limitations documented (at least 3)
- Future improvements listed

Video

- 3-5 minutes duration
- Clear audio
- All required sections covered
- Demo shows working system
- Link included in repository

12. Evaluation Rubric

Total Points: 100

1. RAG Architecture (30 points)

Criteria	Points	Description
Document Ingestion	8	PDF extraction, cleaning, metadata handling
Chunking Implementation	7	At least 2 configurations, proper overlap
Embedding & Vector Store	8	Correct embedding generation, ChromaDB setup
Retrieval Quality	7	Relevant chunks retrieved, proper metadata

2. Prompt Engineering (25 points)

Criteria	Points	Description
Strategy 1: Delimiters	5	Clear instructions, proper formatting
Strategy 2: JSON Output	5	Valid JSON, complete structure
Strategy 3: Few-Shot	5	Quality examples, consistent output
Strategy 4: Chain-of-Thought	5	Logical reasoning, hidden reasoning
Comparison Analysis	5	Documented comparison with insights

3. Web Interface (20 points)

Criteria	Points	Description
Chat Interface	5	Functional, conversation memory

Paper Browser	5	All papers shown, searchable
Citations Display	5	APA format, linked to source
Visualizations/Filters	5	At least 1 chart, working filters

4. Code Quality & Documentation (15 points)

Criteria	Points	Description
Repository Structure	4	Follows required structure
README Completeness	4	All sections present and detailed
Code Quality	4	Clean, documented, error handling
Reproducibility	3	Works with single command

5. Video Presentation (10 points)

Criteria	Points	Description
Content Coverage	4	All required sections
Demo Quality	3	Shows working system clearly
Communication	3	Clear explanation, good pacing

13. Appendix: Resources and Examples

13.1 Useful Documentation

OpenAI

- [OpenAI API Documentation](#)
- [Embeddings Guide](#)
- [Chat Completions](#)

Vector Databases

- [ChromaDB Documentation](#)
- [FAISS GitHub](#)

Web Frameworks

- [Streamlit Documentation](#)
- [Gradio Documentation](#)

PDF Processing

- [PyMuPDF Documentation](#)
- [pdfplumber GitHub](#)

13.2 Paper Sources

Find academic papers at:

- [arXiv](#) - Free preprints
- [Semantic Scholar](#) - AI-powered search
- [Google Scholar](#) - Comprehensive search
- [PubMed](#) - Medical/biological papers
- [IEEE Xplore](#) - Engineering papers
- [ACM Digital Library](#) - Computer science papers

13.3 Example Questions for Evaluation

Create at least 20 evaluation questions. Examples:

Factual Questions

1. "What is the main contribution of [Paper X]?"
2. "In what year was [concept Y] introduced?"
3. "Who are the authors of [Paper Z]?"

Analytical Questions

4. "How does [Method A] compare to [Method B]?"
5. "What are the limitations of [Approach X]?"
6. "What evidence supports [Claim Y]?"

Synthesis Questions

7. "What common themes appear across these papers?"
8. "How has [Topic X] evolved based on these papers?"
9. "What gaps in research do these papers identify?"

Edge Cases

10. "What is the capital of France?" (Out of scope - should say "not in papers")
11. "Summarize a paper I haven't uploaded" (Should handle gracefully)

13.4 Cost Estimation

OpenAI API Costs (Approximate)

Operation	Model	Estimated Cost
Embed 20 papers (~100k tokens)	text-embedding-3-small	~\$0.02
50 queries (avg 2k tokens each)	gpt-4-turbo	~\$3-5
Total project development	-	~\$5-15

Tip: Use GPT-3.5-turbo during development to save costs, then switch to GPT-4 for final testing.

Submission Instructions

1. Push your complete repository to GitHub
2. Ensure the repository is public (or share access with instructor)
3. Upload your video to YouTube (unlisted) or Google Drive
4. Submit the following via [your submission platform]:
 - o GitHub repository URL
 - o Video link
 - o Brief summary (100 words) of your topic

